

First look of daily Minimum Temperature from GHCND data

Tmin data flagged by any of the following 'D|G|K|L|M|N|O|R|S|T|W|X|Z' are removed prior to plotting the timeseries.

```
import xarray as xr
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import glob
import scipy
import cftime
import cartopy
import cartopy.crs as ccrs
import matplotlib.colors as colors
import cmaps
from xhistogram.xarray import histogram
import geopandas as gpd
from matplotlib import cm
import functions_utils

import warnings
warnings.filterwarnings('ignore')

state_borders = \
cartopy.feature.NaturalEarthFeature(category='cultural', \
    name='admin_1_states_provinces_lakes', scale='50m', facecolor='none')

lon_min=185;lon_max=232
lat_min=50;lat_max=72
```

READ STATIONS DATA

QFLAG

```
Blank = did not fail any quality assurance check.
D      = failed duplicate check.
G      = failed gap check.
I      = failed internal consistency check.
K      = failed streak/frequent-value check.
L      = failed check on length of multiday period.
```

```

M      = failed megaconsistency check.
N      = failed naught check.
O      = failed climatological outlier check.
R      = failed lagged range check.
S      = failed spatial consistency check.
T      = failed temporal consistency check.
W      = temperature too warm for snow.
X      = failed bounds check.
Z      = flagged as a result of an official Datzilla
        investigation

QFLAGSregex = ['D|G|K|L|M|N|O|R|S|T|W|X|Z']

var='TMIN'
ifile='/Projects/RAPprototype/GHCND/GHCND_Tmin_Tmax_30yr_AK.nc'
dso = xr.open_dataset(ifile)
flagged=dso[f'{var}_ATTRIBUTES'].str.contains(QFLAGSregex)
dso[var] = dso[var]/10. # degreeC instead of 10th of degreeC
dso[var] = dso[var].where(flagged==False) # remove FLAGGED data

print(dso[var].min().data,dso[var].max().data)

-60.0 37.8

```

Define PP: percentage of presence and PO: percentage of outliers (3sigma)

```

PP = xr.zeros_like(dso['station'])
PO = xr.zeros_like(dso['station'])

for ip in range(0,len(dso['station'])):
    yyy = dso[var].isel(station=ip)
    XXX = np.where(np.isnan(yyy)==False)[0]
    yyyy = functions_utils.remove_outliers_seasons(yyy,3)
    XXXX = np.where(np.isnan(yyyy)==False)[0]
    if len(XXX)>1:
        denom=len(dso[var].isel(station=ip).time[XXX[0]:XXX[-1]+1])
        PP.loc[{'station':dso['station'][ip]}] = len(XXX)/denom*100.
        PO.loc[{'station':dso['station'][ip]}] = (1-len(XXXX)/len(XXX))*100.

```

Get name of stations

```

Tstation = list(dso.station.data)

```

```

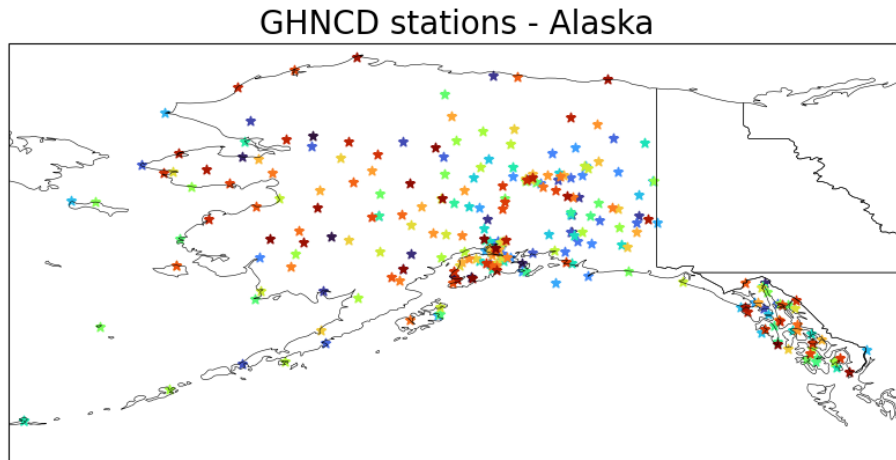
cmap=cmeps.BlGrYeOrReVi200
proj=ccrs.PlateCarree()
colors=cm.get_cmap('turbo',len(dso['station']))
#colors=cm.get_cmap('turbo',10)

fig, ax = plt.subplots(nrows=1,ncols=1,figsize=(10,11),subplot_kw={'projection':proj})

for ip in range(0,len(dso['station'])):
    #for ip in range(30,40):
        ax.scatter(dso['longitude'][ip],dso['latitude'][ip],marker='*',color=colors(ip),label=dso['station'][ip])

ax.set_extent([lon_min,lon_max,lat_min,lat_max],proj)
ax.add_feature(state_borders,zorder=100,linewidth=0.4,edgecolor='k')
handles, labels = ax.get_legend_handles_labels()
#fig.legend(handles, labels, loc='lower center',ncol=6,bbox_to_anchor=(0.5,-0.02),prop={'size':10})
ax.set_title('GHNCD stations - Alaska',fontsize=20)
Text(0.5, 1.0, 'GHNCD stations - Alaska')

```



TIMESERIES by 10 stations

```

k=1
for ii in range(0,len(dso.station),10):
    fig, ax = plt.subplots(nrows=10,ncols=1,figsize=(16,18))
    colors=cm.get_cmap('turbo',11)
    fig.subplots_adjust(hspace=1.15)
    ax_inset = fig.add_axes([0.6, 0.86, 0.3, 0.25], projection=ccrs.PlateCarree())
    ax_inset.set_extent([lon_min,lon_max,lat_min,lat_max],proj)
    ax_inset.add_feature(state_borders,zorder=100,linewidth=0.4,edgecolor='k')

```

```

for ip in range(ii,ii+10):
    ix=ip-ii
    if ip< len(dso.station):
        istation = dso.station[ip].data.item()
        iname = dso.name[ip].data.item().split(',')[0]
        if dso['longitude'][ip]+360<=lon_max:
            ax_inset.scatter(dso['longitude'][ip],dso['latitude'][ip],marker='+',s=10,color='k')
            ax_inset.text(dso['longitude'][ip],dso['latitude'][ip],ix,color=colors(ix),fontsize=
yyy = dso[var].isel(station=ip)
ax[ix].plot(dso['time'],yyy,color=colors(ix),marker='o',markersize=1)
ax[ix].set_ylabel(f'$\degree$C',fontsize=16)
ax[ix].set_xticklabels(ax[ix].get_xticklabels(),fontsize=12)
ax[ix].set_xlabel('',fontsize=16)
ax[ix].grid()
ax[ix].set_title(f'{ix}, {istation}, {iname}, PP={PP.isel(station=ip).data.item():.1f}
ax[ix].set_xlabel('Time',fontsize=16)
fig.text(0.1,1,f'page {k}',fontsize=20)
k = k+1
plt.show()

```

